دليل تقني وتشغيلي شامل

مشروع روبوت حجز مواعيد المستشفى عبر واتساب

شرح بنية المشروع، وظيفة كل ملف، قاعدة البيانات، تدفق الرسائل، الحجز والتعديل والإلغاء، التذكيرات، لوحة التحكم، التكامل مع Meta، وطريقة التشغيل.

المشروع: `hospital_chatbot`

المسار: `e:\4S\hospital_chatbot`

تاريخ إعداد الدليل: 16 يوليو 2026

فهرس المحتويات

1. الفكرة العامة وحالة المشروع
2. التقنيات المستخدمة
3. المعمارية العامة
4. هيكل المجلدات
5. التشغيل ونقاط الدخول
6. رحلة رسالة واتساب
7. فلو الحجز الكامل
8. التعديل والإلغاء
9. الساعات والـ Slots والعطلات
10. التذكيرات
11. الرسائل الصوتية
12. السلامة والتحويل للبشر
13. قاعدة البيانات والعلاقات
14. لوحة التحكم
15. دليل كل ملف
16. متغيرات البيئة
17. أوامر التشغيل
18. ملاحظات وحدود حالية

1. الفكرة العامة وحالة المشروع

المشروع Backend متكامل لروبوت مستشفى يعمل داخل WhatsApp Cloud API. المريض يستطيع اختيار اللغة، اختيار تخصص أو طبيب، مشاهدة كروت الأطباء، اختيار تاريخ ووقت، تأكيد الحجز، تعديل الموعد أو إلغاؤه. جميع العمليات الأساسية تُحفظ في PostgreSQL وتظهر في لوحة Streamlit للإدارة.

> **المسار الفعلي المعتمد حالياً:** تشغيل `uvicorn app.main:app` . الملفات الموجودة في جذر المشروع مثل `main.py` و `main_ai.py` هي تجارب أقدم وليست نقطة التشغيل الإنتاجية الحالية.

واجهة المريض

- واتساب: نصوص، أزرار، قوائم، كروت صور، WhatsApp Flow اختياري.
- دعم العربية والإنجليزية.
- حجز، تعديل، إلغاء، استعلام، تحويل للاستقبال.
- رسائل صوتية مع تحويل الكلام إلى نص.

واجهة الإدارة

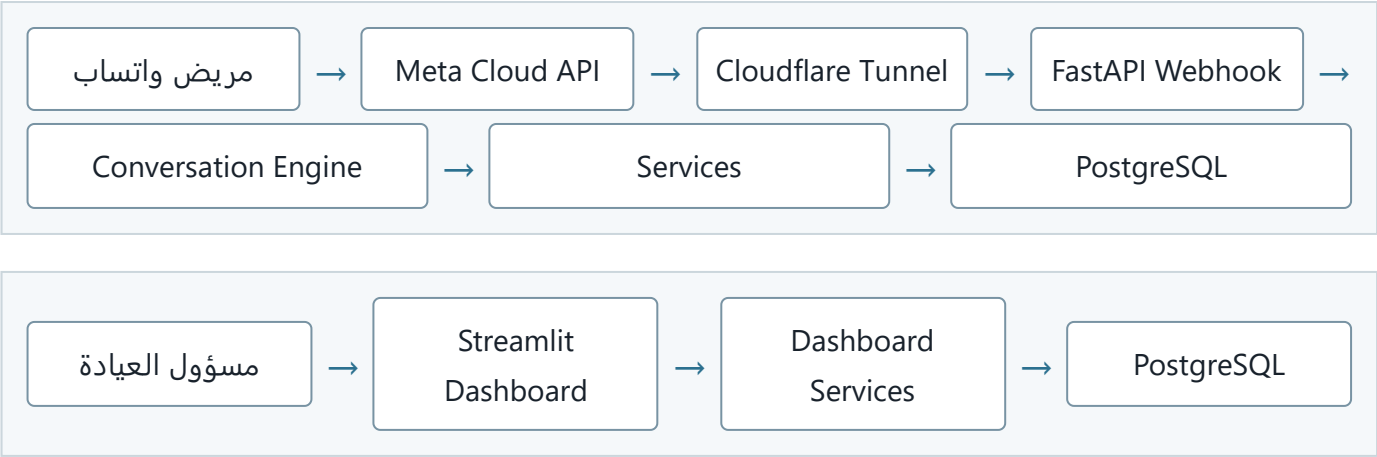
- Streamlit Dashboard.
- إدارة العيادة، الأقسام، الأطباء، ساعات العمل والعطلات.
- المواعيد، سجل التغييرات، المحادثات والتذكيرات.
- KPIs، الاستخدام والتكلفة التقديرية.

2. التقنيات المستخدمة

الطبقة	التقنية	الدور
API	`FastAPI + Uvicorn`	Webhook واتساب وREST API التجريبي.

قاعدة البيانات	`PostgreSQL + SQLAlchemy 2`	تخزين المرضى والأطباء والمواعيد والمحادثات والتذكيرات.
إدارة الحوار	`LangGraph`	تشغيل عقدتي حفظ الرسالة ثم معالجة الحالة.
واتساب	`Meta WhatsApp Cloud API`	إرسال واستقبال النصوص والأزرار والقوائم والكروت والـ Flow.
الذكاء الاصطناعي	`OpenAI / Groq`	حل الاختيارات الغامضة وSTT للصوت؛ الفلو الأساسي يعتمد قواعد واضحة.
الصوت	`Groq Whisper + edge-tts / gTTS`	تحويل الصوت إلى نص، مع أدوات TTS موجودة بالملفات الجذرية.
لوحة الإدارة	`Streamlit + Pandas`	إدارة البيانات وعرض الجداول والتقارير.
النفق	`Cloudflared`	إتاحة localhost بعنوان HTTPS ليصل إليه Meta Webhook.

3. المعمارية العامة



تقسيم المسؤوليات

المجلد	مسؤوليته
`app/api/routes`	استقبال HTTP، استخراج بيانات Meta، وإرسال العمل إلى الخدمات.
`app/graph`	إدارة حالة المحادثة والفلو التفاعلي خطوة بخطوة.
`app/services`	منطق العمل: الحجز، الـ slots، الأطباء، المرضى، التذكيرات، واتساب، التحويل.
`app/models`	تعريف جداول SQLAlchemy وعلاقات قاعدة البيانات.
`app/core`	الإعدادات، الترجمة، الرسائل، الوقت، السلامة والـ prompts.
`dashboard`	واجهة الإدارة وخدمات العرض.
`app/scripts`	زرع بيانات الديمو وتشغيل التذكيرات يدوياً أو مجدولاً.

4. هيكل المجلدات

```
hospital_chatbot/
├── app/
│   ├── main.py                نقطة تشغيل FastAPI الفعلية
│   ├── api/routes/            Webhook + Booking REST API
│   ├── graph/                 محرك المحادثة LangGraph
│   ├── core/                  Config, i18n, safety, time
│   ├── services/              منطق النظام والخدمات
│   ├── models/                جداول SQLAlchemy
│   ├── db/                    الاتصال والـ enums
│   ├── scripts/               seed + reminders
│   ├── flows/                 WhatsApp Flow JSON
│   └── schemas/               حزمة محجوزة للتوسّع
├── dashboard/
│   ├── streamlit_app.py       لوحة الإدارة
│   └── ui_i18n.py             ترجمة واجهة الإدارة
├── docs/                      وثائق التشغيل وهذا الدليل
├── requirements.txt           مكتبات بايثون
├── .env / .env.example        أسرار وإعدادات البيئة
├── README.md                  تعريف مختصر
├── voice_processor.py         STT الفعلي الذي تستدعيه الخدمة
├── main.py                    نموذج قديم غير معتمد
├── main_ai.py                 نموذج Groq قديم مستقل
└── test.py                    نموذج صوت محلي قديم
```

5. التشغيل ونقاط الدخول

API المعتمد

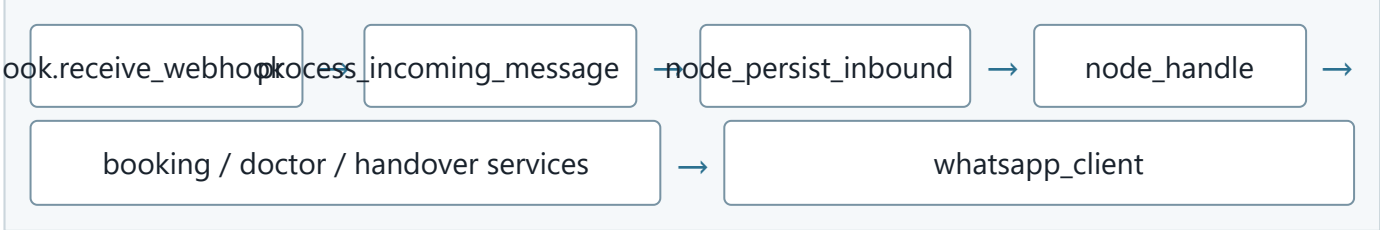
الملف `app/main.py` ينشئ تطبيق FastAPI، ويشغّل `create_tables()` عند البداية، ثم يضمّن `webhook_router` و `booking_router`.

- `GET /` : فحص أن السيرفر يعمل.
- handshake : `GET /webhook` للتحقق من Meta.
- `POST /webhook` : استقبال رسائل وحالات التسليم.
- `GET /booking/departments` : قائمة التخصصات.
- `GET /booking/doctors` : قائمة الأطباء.
- `GET /booking/doctors/{id}/slots` : الأوقات المتاحة.
- `POST /booking/confirm` : تأكيد مباشر للاختبار بدون واتساب.
- Swagger UI : `GET /docs` من FastAPI.

> **قاعدة البيانات:** `app/db/database.py` يبني Engine بـ connection pool، ويوفّر `SessionLocal` وDependency اسمها `get_db`. إنشاء الجداول بـ `Base.metadata.create_all` مناسب للتطوير؛ الإنتاج يفضّل Alembic migrations.

6. رحلة رسالة واتساب بين الملفات

1. المريض يرسل رسالة إلى رقم WhatsApp Business.
2. Meta يرسل JSON إلى `POST /webhook` في `app/api/routes/webhook.py` .
3. `receive_webhook` يحدد: نص، زر، قائمة، Flow، صوت، أو status update.
4. النص/الزر يُرسل كـ Background Task إلى `_handle_message` .
5. `_handle_message` يفتح DB session ويستدعي `process_incoming_message` .
6. `conversation_engine.get_graph()` يشغّل LangGraph.
7. العقدة `node_persist_inbound` تحفظ Conversation وMessage.
8. العقدة `node_handle` تقرأ حالة المحادثة وتنفذ الخطوة المناسبة.
9. الخدمات تقرأ/تكتب PostgreSQL ثم `whatsapp_client.py` يرسل رد Meta.



حالات المحادثة

مخزنة في `Conversation.state` ومُعرّفة داخل `ConversationState` :

`confirming` `patient_details` `booking_slot` `booking_date` `booking_doctor` `main_menu` `language_select` `new` `closed` `handover` `cancel` `reschedule`

الحالة الأساسية محفوظة في DB، لكن تفاصيل draft أثناء الحجز موجودة في قاموس الذاكرة `_SESSION` داخل محرك المحادثة، مع آلية استرجاع/إعادة بداية عند فقدها بعد reload.

7. فلو الحجز الكامل



7.1 اختيار التخصص والطبيب

- `intent_service.detect_intent` يتعرف على نية الحجز بالقواعد أولاً.
- `doctor_service.list_departments` يجلب الأقسام؛ القسم هو التخصص الظاهر للمريض.
- `doctor_service.list_doctors_by_department` يجلب أطباء التخصص.
- `doctor_card_payload` يحضّر الاسم والصورة والتخصص والنبذة.
- `whatsapp_client.send_doctor_cards` يرسل الكروت تباعاً (حتى 5).
- إذا كان WhatsApp Flow مفعّلاً، `flow_service` يبني Carousel + radio list؛ عند الفشل يعود للكروت.

7.2 توليد وعرض الأوقات

- `booking_service.get_available_dates_and_slots` يتحقق من الطبيب ويفك أي holds منتهية.
- `slot_service.ensure_slots_for_doctor` يولّد slots من ساعات الطبيب الأسبوعية.
- يتجاوز الماضي، الحد الأدنى قبل الحجز، عطلات الطبيب، وعطلات العيادة.
- `list_available_slots` يعيد فقط الحالة `available` لنفس الطبيب والعيادة.
- `group_slots_by_date` يجمعها حسب اليوم لتظهر في قوائم واتساب.

7.3 الحجز المؤقت والتأكيد

1. `hold_slot` يقفل صف الـ slot بـ `SELECT FOR UPDATE`.
2. يغير الحالة من `available` إلى `held` لمدة افتراضية 3 دقائق.
3. يربط الـ hold برقم واتساب داخل marker في `updated_by`.
4. المريض يكتب الاسم، وMRN/DOB إذا كانت مطلوبة من الإعدادات.
5. يعرض البوت الملخص ثم ينتظر «تأكيد» أو «إلغاء».
6. `confirm_booking` يقفل الصف مجدداً ويتأكد أن الوقت متاح لنفس الرقم.
7. ينشئ/يحدّث Patient، وينشئ Appointment بحالة `confirmed`.
8. يغيّر Slot إلى `booked` ويمسح انتهاء الـ hold.
9. ينشئ AppointmentHistory وReminder records ويغلق Conversation.
10. يعمل `commit` ثم يرجع reference مثل `APT-...`.

منع الحجز المزدوج

> الحماية متعددة الطبقات: صف الـ slot يُقفل Transactionally؛ الحالة `booked` لا تظهر ضمن الأوقات المتاحة؛ وهناك partial unique index باسم `uq_appointment_active_slot` يمنع وجود أكثر من Appointment نشط لنفس slot. عند محاولتين متزامنتين، الثانية ترى الحالة محجوزة وتفشل.

8. تعديل الموعد وإلغاؤه

التعديل Reschedule

1. `list_upcoming_appointments_for_phone` يعرض المواعيد المؤكدة التي تخص رقم المريض فقط.
2. المريض يختار الموعد ثم الطبيب/التاريخ/الوقت الجديد.
3. الوقت الجديد يوضع في hold لنفس رقم واتساب.
4. `reschedule_appointment` يتحقق من ملكية الموعد ومن cutoff.
5. الـ slot القديم يصبح `available` والجديد يصبح `booked` .
6. يتحدث Appointment ويُضاف سجل في AppointmentHistory.
7. التذكيرات القديمة تُلغى وتُنشأ تذكيرات للوقت الجديد.

الإلغاء Cancel

1. المريض يختار موعده ويؤكد الإلغاء.
2. `cancel_appointment` يتحقق من الهاتف والموعد وحالة confirmed.
3. Appointment تصبح `cancelled` .
4. Slot تصبح `available` فتظهر مجدداً لمريض آخر.
5. يُكتب AppointmentHistory وتُلغى reminders المعلقة.

> الإلغاء والتعديل يخضعان لـ `CANCEL_CUTOFF_HOURS` و `RESCHEDULE_CUTOFF_HOURS` . إذا اقترب الموعد أكثر من المسموح، ينتقل الطلب للاستقبال.

9. ساعات العمل، الأوقات، والعطلات

ساعات العمل الأسبوعية

`DoctorWorkingHours` يحدد يوم الأسبوع (0=الإثنين...6=الأحد)، وقت البداية والنهاية، طول الموعد، والحد الأقصى. لوحة الإدارة تحفظ الساعات عبر `admin_dashboard_service.upsert_working_hours` ثم تولّد slots تلقائياً.

عطلة الطبيب

`DoctorUnavailability` يغلق طبيباً محدداً بين تاريخين/وقتين. عند إضافتها من لوحة الإدارة، الأوقات المتاحة المتداخلة تتحول إلى `blocked` .

عطلة العيادة

`ClinicHoliday` يغلق العيادة كلها. `block_available_slots_for_range` يمنع الـ slots المتاحة لكل الأطباء في النطاق.

حالات الـ Slot

الحالة	المعنى	هل تظهر للحجز؟
`available`	متاحة.	نعم.
`held`	محجوزة مؤقتاً أثناء إدخال البيانات.	لا.
`booked`	موعد مؤكد.	لا، حتى الإلغاء/التعديل.
`blocked`	عطلة أو إغلاق.	لا.
`expired`	قيمة معرفة للتوسع؛ لا تُستخدم حالياً بشكل أساسي.	لا.

10. التذكيرات

- `create_booking_reminders` ينشئ سجلات pending قبل الموعد (افتراضياً 24 و2 ساعة).
- `process_due_reminders` يجلب المستحق ويرسل رسالة واتساب مترجمة.
- الـ dry-run يعرض ما سيُرسل بدون إرسال حقيقي.
- `cancel_pending_reminders` يلغي المعلق عند إلغاء/تعديل الموعد.
- Webhook statuses يحدّث reminder إلى delivered/read/failed حسب Meta receipts.
- لوحة الإدارة تعرض السجلات وتسمح بالتشغيل التجريبي وإعادة الفاشل.

التشغيل اليدوي يتم من `app/scripts/run_reminders.py`، ويمكن جدولة الأمر عبر Task Scheduler أو cron.

11. الرسائل الصوتية

1. Webhook يكتشف `message.type = audio` ويستخرج `media_id`.
2. `voice_service.transcribe_whatsapp_audio` يستدعي `voice_processor.process_whatsapp_audio`.
3. يُنزّل الصوت من Meta ويرسله إلى Groq Whisper ثم يحذف الملف المؤقت.
4. يُحفظ Message من نوع voice وVoiceMessage metadata داخل DB.
5. إذا كانت الثقة منخفضة، يطلب البوت إعادة التسجيل/الكتابة.
6. إذا كانت جيدة، يعرض النص ويسأل المستخدم «هل هذا صحيح؟» قبل متابعة الفلو.

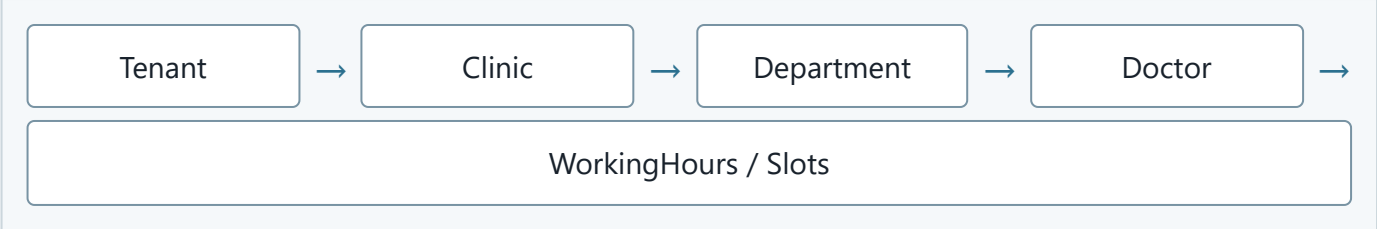
> `test.py` يحتوي نموذجاً قديماً لـ EgypTalk local ASR، بينما المسار الحالي من `app/services/voice_service.py` يستورد `voice_processor.py` الذي يستخدم Groq Whisper.

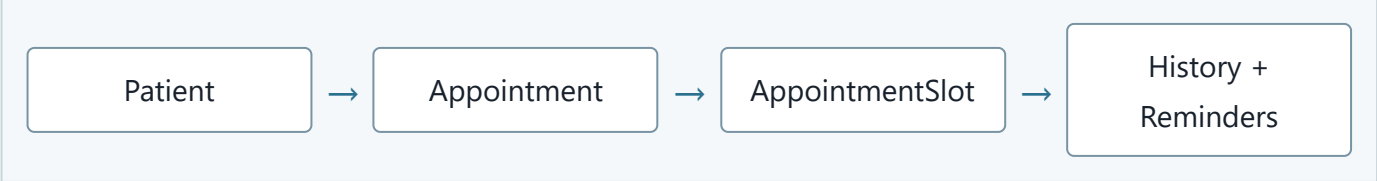
12. السلامة والتحويل للاستقبال

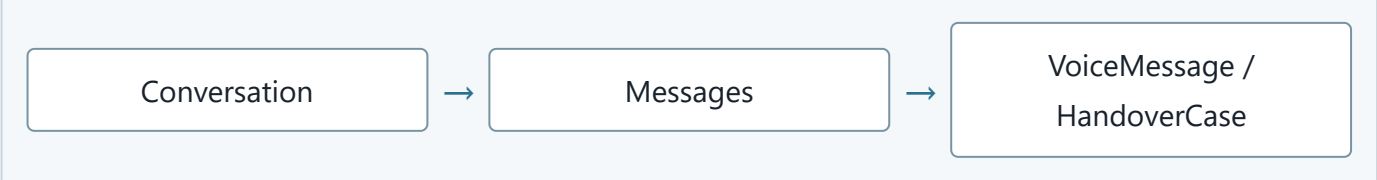
- `core/safety.py` يتعرف على عبارات طبية عالية الخطورة ولا يقدم تشخيصاً.
- يرسل رسالة طوارئ مناسبة للغة ويُنشئ HandoverCase عاجلة.
- `handover_service.create_handover` يربط الحالة بالمحادثة والمريض إن وجد.

- يحفظ سبب التحويل، الأولوية، حالة open، وملخص آخر الرسائل.
- إذا كان رقم الاستقبال مضبوطاً، يمكن إشعار الاستقبال عبر واتساب.
- أخطاء Webhook غير المتوقعة تتحول أيضاً إلى handover أولوية عالية.

13. قاعدة البيانات والعلاقات







الجدول / الموديل	وظيفته والعلاقات المهمة
`Tenant`	الجهة المالكة؛ لها Clinics وAdmins وDoctors.
`Clinic`	العيادة؛ تربط الأقسام والأطباء والمرضى والمواعيد والمحادثات والإعدادات.
`Department`	القسم/التخصص الطبي داخل العيادة؛ يحتوي أطباء.
`ClinicHoliday`	إغلاق عام للعيادة خلال نطاق زمني.
`Doctor`	بيانات الطبيب وصورته وتخصصه وعلاقاته بالساعات والـ slots والمواعيد.
`DoctorLocation`	مواقع/غرف الطبيب وساعات الموقع.
`DoctorWorkingHours`	جدول أسبوعي متكرر لتوليد المواعيد.
`DoctorUnavailability`	إجازة/حظر خاص بالطبيب.
`Patient`	رقم واتساب، الاسم، MRN، DOB، ومواعيده ومحادثاته.
`AppointmentSlot`	وقت محدد لطبيب؛ له status وhold expiry.
`Appointment`	الحجز الذي يربط المريض والطبيب والـ slot ويحمل reference وstatus.
`AppointmentHistory`	Audit trail للتأكيد والتعديل والإلغاء وتغير الـ slots.
`Conversation`	حالة محادثة رقم واتساب واللغة والنية الحالية.
`Message`	سجل inbound/outbound وحالة التسليم ومعرف Meta.
`VoiceMessage`	تفاصيل الصوت، confidence، provider، transcript.

`Reminder`	تذكير مرتبط بموعد ووقت إرسال وحالة ومحاولات.
`HandoverCase`	طلب تحويل لموظف بشري مع السبب والأولوية والملخص.
`AdminUser`	مستخدم إدارة مرتبط بـ Tenant.
`ConfigSetting`	إعدادات لكل عيادة Key/value.
`UsageLog`	مزود/عملية/tokens/cost للتقارير.

حالات Appointment

draft | held | confirmed | rescheduled | cancelled | completed | no_show | pending_handover

14. لوحة التحكم

`dashboard/streamlit_app.py` هو تطبيق Streamlit متعدد الشاشات. يفتح DB session، يستدعي `admin_dashboard_service` و `dashboard_service`، ثم يعرض النتائج بـ Pandas.

الشاشة	ما تفعله
KPIs	محاولات الحجز، المؤكد، الإلغاء، التعديل، handover، no-show، الصوت والتكلفة.
Clinic setup	اسم العيادة، واتساب، timezone، اللغة، العنوان، إعدادات عامة وعطلات العيادة.
Departments	إضافة/تعديل التخصصات.
Doctors	إضافة/تعديل/تعطيل/حذف الطبيب وصورته وقسمه وتخصصه.
Working hours	ساعات أسبوعية للطبيب + إجازاته الخاصة؛ توليد وحظر slots.
Doctor schedule	لكل طبيب: Free، Booked، Held، Blocked مع بيانات الحجوزات.
Appointments	القائمة، الحالة، الإلغاء، التعديل، no-show، وسجل التاريخ.
Conversations	المحادثات والرسائل الكاملة وhandover cases.
Reminders	السجلات، إرسال المستحق، force demo، وإعادة الفاشل.
Settings	ساعات التذكير، اسم template، رقم التصعيد، consent، fallback.
Usage / costs	ملخص الاستخدام والتكلفة حسب المزود/العملية والسجلات الحديثة.

`dashboard/ui_i18n.py` يحتوي قاموس النصوص العربي/الإنجليزي وترتيب الشاشات في `SCREENS`. الوقت يُعرض بنظام 12 ساعة عن طريق `core/time_format.py`.

15. دليل كل ملف في المشروع

الرمز **فعّال** يعني داخل المسار الحالي؛ **قديم/مساند** يعني تجربة سابقة أو ملف غير داخل التشغيل الأساسي.

15.1 ملفات الجذر

الملف	الحالة	الوظيفة
`README.md`	وثيقة	تعريف مختصر جداً بالمشروع والتقنيات.
`requirements.txt`	**فعّال**	اعتمادات FastAPI، SQLAlchemy، LangGraph، Streamlit، OpenAI/Groq والصوت.
`.env`	**فعّال/سري**	القيم الحقيقية للمفاتيح والاتصال؛ لا يُنشر ولا يُطبع في الوثائق.
`.env.example`	قالب	أسماء الإعدادات المطلوبة بدون أسرار.
`voice_processor.py`	**مساند فعلي**	تحميل صوت واتساب، Groq Whisper STT، وأدوات TTS ورفع/إرسال الصوت.
`main.py`	**قديم**	Prototype قديم يعتمد sessions داخل الذاكرة وبيانات أطباء hard-coded؛ لا يشغّل النظام الحالي.
`main_ai.py`	**قديم**	Prototype مستقل يستخدم Groq function calling وقاموس أطباء وهمياً؛ ليس PostgreSQL الحقيقي.
`test.py`	**قديم**	نسخة تجربة EgypTalk local ASR + TTS؛ ليست مسار الصوت الحالي.

15.2 نقطة التشغيل وAPI

الملف	الوظيفة
`app/main.py`	ينشئ FastAPI، ينشئ الجداول عند startup، ويسجل routers.
`app/api/routes/__init__.py`	يصدّر `booking_router` و `webhook_router` .
`app/api/routes/webhook.py`	تحقق Meta، استقبال الرسائل، استخراج أنواع الإدخال، background tasks، الصوت، receipts، Flow، ومعالجة الخطأ.
`app/api/routes/booking.py`	REST endpoints للتخصصات والأطباء والـ slots والتأكيد؛ مفيدة للاختبار بدون واتساب.

15.3 محرك المحادثة

الملف	الوظيفة
`app/graph/__init__.py`	واجهة الحزمة: تصدير معالجة الرسالة وإكمال اختيار الطبيب من Flow.
`app/graph/conversation_engine.py`	قلب النظام: GraphState، session draft، اللغة والقائمة، resolving، حالات الحجز والتعديل والإلغاء، بناء LangGraph وتشغيله.

15.4 ملفات Core

الملف	الوظيفة
`app/core/config.py`	قراءة .env وإعدادات DB، AI، WhatsApp، العيادة، سياسات الحجز والتذكيرات.
`app/core/i18n_bot.py`	طبقة النصوص ثنائية اللغة، توطين الألقاب والتخصصات، cards، confirmations والقوائم.
`app/core/messages.py`	الدوال العربية الأساسية للترحيب، القائمة، التأكيد، التعديل، الإلغاء والتذكير.
`app/core/time_format.py`	توحيد الساعة 12h بالعربية/الإنجليزية، parsing وخيارات الوقت للداشبورد.
`app/core/safety.py`	كشف العبارات الطبية عالية الخطورة ورسائل الطوارئ وعدم تقديم نصيحة طبية.
`app/core/prompts.py`	**System prompt قديم لموظف استقبال AI؛ المسار الحالي guided flow لا يعتمد عليه مباشرة.**
`app/core/memory.py`	**قواميس ذاكرة/منع تكرار قديمة؛ محرك المحادثة الحالي لديه session خاص به وDB logs.**
`app/core/__init__.py`	تعريف package.

15.5 خدمات منطق العمل

الملف	الوظيفة وأهم الدوال
`app/services/booking_service.py`	الحجز transactionally: availability، hold، confirm، duplicate warning، upcoming، reschedule، cancel، expired holds، no-show.
`app/services/slot_service.py`	توليد الـ slots من ساعات العمل، تطبيق lead time والعطلات، عرض available، التجميع والقفل FOR UPDATE.
`app/services/doctor_service.py`	البحث عن الأقسام/الأطباء، soft matching، وجعل بيانات الطبيب مناسبة للكرت.
`app/services/patient_service.py`	إيجاد أو إنشاء Patient حسب العيادة ورقم واتساب وتحديث بياناته.
`app/services/conversation_service.py`	إنشاء/استرجاع Conversation، تحديث اللغة والحالة، حفظ Message وVoice metadata.
`app/services/whatsapp_client.py`	عميل Meta: POST موحد، نص، أزرار، list، doctor card(s)، وWhatsApp Flow.
`app/services/intent_service.py`	Intent enum وكشف النية بالقواعد؛ fallback إلى LLM عند توفر OpenAI.
`app/services/choice_resolver.py`	حل النص الحر إلى option id: normalization وقواعد ثم LLM fallback.
`app/services/flow_service.py`	إعداد بيانات Doctor ImageCarousel، تنزيل/ضغط الصور parse، Base64 إكمال Flow.
`app/services/reminder_service.py`	إنشاء/إلغاء/استعلام/إرسال التذكيرات وتسجيل wamid والحالة.
`app/services/handover_service.py`	إنشاء حالات التحويل، transcript snippet، أولوية وإخطار الاستقبال، وقائمة الحالات.
`app/services/voice_service.py`	Adapter بسيط إلى voice_processor مع confidence heuristic.
`app/services/dashboard_service.py`	Queries للموعد والتاريخ والمحادثات والرسائل والتذكيرات وجدول الطبيب والإلغاء.

app/services/admin_dashboard_service.py	CRUD العيادة/الأقسام/الأطباء/الساعات/العطل، KPIs، usage، reschedule، reminders، no-show.
app/services/helpers.py	توليد appointment reference.
app/services/__init__.py	تعريف package للخدمات.

15.6 قاعدة البيانات والمودلز

الملف	الوظيفة
app/db/database.py	Engine، pool، SessionLocal، Base، get_db، create_tables.
app/db/enums.py	Statuses للحجز والـ slot والتذكير والمحادثة والرسالة والأولوية واللغة.
app/db/__init__.py	تعريف package.
app/models/__init__.py	يستورد كل المودلز لتسجيلها في Base.metadata.
app/models/core_models.py	Tenant، Clinic، Department، ClinicHoliday.
app/models/doctor_models.py	Doctor، DoctorLocation، DoctorWorkingHours، DoctorUnavailability.
app/models/patient_models.py	Patient وعلاقاته بالمواعيد والمحادثات.
app/models/appointment_models.py	AppointmentSlot، Appointment، AppointmentHistory، القيود والفهارس.
app/models/conversation_models.py	Conversation، Message، VoiceMessage.
app/models/reminder_models.py	Reminder، schedule/status/Meta id/retries.
app/models/handover_models.py	HandoverCase والسبب والأولوية والموظف والحالة.
app/models/admin_models.py	AdminUser وConfigSetting.
app/models/usage_models.py	UsageLog للتوكنز والتكلفة والمزود والعملية.

15.7 السكربتات، الـ Flow، والداشبورد

الملف	الوظيفة
app/scripts/seed_demo.py	إنشاء Tenant/Clinic/Cardiology و4 أطباء بصور وساعات 30 دقيقة وAdmin وConfig. الداتا الحالية قد تحتوي إضافات من الداشبورد.
app/scripts/run_reminders.py	CLI للتذكيرات: force، dry-run، limit.
app/scripts/__init__.py	تعريف package.
app/flows/doctor_picker.json	تعريف شاشة WhatsApp Flow للصور والاختيار.
app/flows/__init__.py	تعريف package.
app/schemas/__init__.py	حزمة فارغة/محجوزة لإضافة Pydantic schemas لاحقاً.
dashboard/streamlit_app.py	كل صفحات لوحة الإدارة، login، CSV export، display formatting، forms، والتنقل.

`dashboard/ui_i18n.py`	قاموس ترجمة لوحة الإدارة وقائمة الشاشات.
`docs/WHATSAPP_FLOW_SETUP.md`	خطوات إنشاء ونشر Flow في Meta وتفعيل متغيراته.

16. متغيرات البيئة

المتغير	الغرض
`DATABASE_URL`	رابط PostgreSQL.
`DEFAULT_TENANT_ID / DEFAULT_CLINIC_ID`	سياق الـ MVP أحادي العيادة.
`VERIFY_TOKEN`	تحقق Meta Webhook.
`WHATSAPP_TOKEN / PHONE_NUMBER_ID`	إرسال الرسائل عبر Cloud API.
`USE_DOCTOR_FLOW / WHATSAPP_FLOW_ID / CTA / DRAFT`	تفعيل WhatsApp Flow الاختياري.
`OPENAI_API_KEY / GROQ_API_KEY / OPENAI_MODEL`	LLM وSTT.
`SLOT_HOLD_MINUTES`	مدة الحجز المؤقت.
`BOOKING_WINDOW_DAYS / MIN_LEAD_HOURS`	نافذة الأيام والحد الأدنى قبل الموعد.
`CANCEL_CUTOFF_HOURS / RESCHEDULE_CUTOFF_HOURS`	منع الإلغاء/التعديل المتأخر.
`DUPLICATE_APPOINTMENT_DAYS`	فترة التحذير من موعد مشابه.
`ALLOW_OVERBOOKING`	Flag موجود؛ النظام عملياً يستخدم slot واحد/حجز واحد.
`REQUIRE_MRN / REQUIRE_DOB`	جعل بيانات المريض إلزامية.
`REMINDER_HOURS_BEFORE`	مثلاً 24,2.
`CLINIC_DISPLAY_NAME / LOCATION / LOCATION_AR`	اسم وموقع العيادة في الرسائل.
`RECEPTIONIST_WHATSAPP`	رقم التصعيد البشري.
`ADMIN_DASHBOARD_PASSWORD`	كلمة مرور Dashboard؛ الافتراضي في الكود admin123 للتطوير.

> **أمان:** لا ترفع `.env`.

17. أوامر التشغيل

تثبيت المكتبات والبيانات

```
cd e:\4S\hospital_chatbot
pip install -r requirements.txt
python -m app.scripts.seed_demo
```

تشغيل FastAPI

```
uvicorn app.main:app --reload --host 127.0.0.1 --port 8000
```

تشغيل لوحة الإدارة (Terminal ثانٍ)

```
streamlit run dashboard/streamlit_app.py
```

تشغيل Tunnel لـ Meta (Terminal ثالث)

```
.\cloudflared tunnel --url http://127.0.0.1:8000
```

التذكيرات

```
python -m app.scripts.run_reminders --dry-run --force
python -m app.scripts.run_reminders --force --limit 2
```

بقاء Uvicorn أو Streamlit أو Cloudflared بدون رجوع لسطر الأوامر طبيعي؛ هذه عمليات Servers طويلة التشغيل.

18. ملاحظات تقنية وحدود حالية

1. **المسار القديم:** لا تشغّل `uvicorn main:app` إذا أردت النظام الحالي؛ استخدم `app.main:app`.
2. **Draft memory:** تفاصيل الحجز المؤقتة ليست كلها DB-backed؛ restart أثناء الفلو قد يعيد المستخدم للقائمة.
3. **Hold ownership:** مخزن حالياً كـ marker داخل `updated_by` وليس عموداً مستقلاً؛ عمود `held_by_phone` سيكون أوضح مستقبلاً.
4. **Timezone:** كثير من المقارنات تستخدم datetimes naive و `utcnow()` رغم وجود Clinic timezone.
5. **30 دقيقة:** الديمو يعتمد 30 دقيقة، لكن لوحة الإدارة تسمح بتغيير طول slot؛ يجب إبقاؤه 30 لقبول الديمو.
6. **Reminder delivery:** الإرسال الحالي session message/buttons؛ Meta templates أنسب خارج نافذة خدمة 24 ساعة.
7. **Alembic:** المكتبة مثبتة لكن لا يظهر مجلد migrations؛ الإنتاج يحتاج migrations بدلاً من create_all فقط.
8. **Auth:** لوحة الإدارة تستخدم مقارنة Password بيئية بسيطة، وليست منظومة hashed passwords/sessions كاملة.
9. **Tests:** لا توجد test suite منظمة للمسار الحالي؛ `test.py` ليس اختباراً آلياً بل prototype صوت.
10. **Legacy cleanup:** فصل الملفات القديمة في مجلد archive يقلل احتمال تشغيل التطبيق الخطأ.

الخلاصة

التصميم الحالي يفصل HTTP عن محرك الحوار وعن منطق العمل وعن قاعدة البيانات بصورة جيدة. أهم مسار هو: `webhook.py → conversation_engine.py → services → models/PostgreSQL → whatsapp_client.py`. لوحة Streamlit تستخدم نفس قاعدة البيانات، ولذلك يظهر الحجز والتعديل والإلغاء فوراً في شاشات الإدارة.